
Software Requirements Specification
for

ShieldAPI

Version 1.0 approved

Prepared by: 1. Anshuman Biswas (Roll No: 2024BCD0008)

2. Nipun Abhilash(Roll No: 2024BCD0024)

3. Syed Zaid Gafar(Roll No: 2024BCD0028)

4. Aman Madhesiya(Roll No: 2024CD0032)

Organization: Indian Institute of Information Technology Kottayam

Date Created: August 4, 2026

Table of Contents

1. Introduction

1.1 Purpose

1.2 Document Conventions

1.3 Intended Audience and Reading Suggestions

1.4 Product Scope

1.5 References

2. Overall Description

2.1 Product Perspective

2.2 Product Functions

2.3 User Classes and Characteristics

2.4 Operating Environment

2.5 Design and Implementation Constraints

2.6 User Documentation

2.7 Assumptions and Dependencies

3. External Interface Requirements

3.1 User Interfaces

3.2 Hardware Interfaces

3.3 Software Interfaces

3.4 Communications Interfaces

4. System Features

4.1 Asynchronous Proxy Routing

4.2 Distributed Token-Bucket Rate Limiting

4.3 Heuristic Anomaly Detection Engine

5. Other Nonfunctional Requirements

5.1 Performance Requirements

5.2 Safety Requirements

5.3 Security Requirements

5.4 Software Quality Attributes

5.5 Business Rules

6 Other Requirements

Appendix A: Glossary

Appendix B: Analysis Models

1. Introduction

1.1 Purpose

This document specifies the software requirements for ShieldAPI, a self-hosted API Gateway equipped with real-time anomaly detection. It details the functional and non-functional requirements necessary to implement the gateway, rate-limiting protocols, and the administrative dashboard. This specification is designed to guide the development phases and provide a baseline for architectural evaluation .

1.2 Document Conventions

This document adheres to the IEEE 830-1998 standard for Software Requirements Specifications. Priority levels for functional requirements are categorized as High (critical to system architecture), Medium (enhances operational efficiency), and Low (UI/UX polish). The keyword "shall" indicates a mandatory requirement.

1.3 Intended Audience and Reading Suggestions

The primary audience includes software architecture evaluators, DevSecOps engineers, and system administrators. Readers evaluating the architectural patterns should focus heavily on Section 2.1 (Product Perspective) and Section 4 (System Features), where the implementation of the Broker, Pipes and Filters, and MVC patterns are detailed.

1.4 Product Scope

Modern distributed systems frequently suffer performance bottlenecks when handling volumetric traffic spikes and targeted malicious probing at the application layer. ShieldAPI completely abstracts defensive logic away from the main application. By operating as a high-performance proxy gate, it inspects, throttles, and filters incoming HTTP requests before they reach fragile backend databases. The objective is to decouple security from business logic,

providing an enterprise-grade infrastructure tool that ensures high availability for downstream microservices.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- FastAPI ASGI Documentation (Python 3.x).
- Redis In-Memory Data Store Specifications.

2. Overall Description

2.1 Product Perspective

ShieldAPI is a self-contained infrastructure product. It is positioned as an intermediary layer between external internet traffic and internal backend microservices. The architecture strictly adheres to three design patterns:

1. **Broker Pattern:** Redis is utilized to manage state (rate limiting and IP blacklists), ensuring the gateway remains stateless and horizontally scalable.
2. **Pipes and Filters Pattern:** The Anomaly Guardian engine processes raw asynchronous access logs as a continuous data stream, passing data through parsing and heuristic threshold filters to identify threats.
3. **Model-View-Controller (MVC):** The administrative dashboard provides immediate visual feedback on system health by decoupling the user interface (React) from the routing logic.

2.2 Product Functions

- **Traffic Interception:** Intercept and route Layer 7 HTTP requests.
- **Request Throttling:** Enforce request quotas per IP address using a token-bucket algorithm.
- **Threat Mitigation:** Continuously stream server logs to detect brute-force attempts and dynamically blacklist malicious actors.
- **Traffic Visualization:** Display real-time gateway health and active blacklist records to administrators.

2.3 User Classes and Characteristics

- **System Administrators (Primary):** Highly technical users requiring real-time visibility into infrastructure health. They interact primarily with the React dashboard to monitor logs and manage the IP blacklist.
- **External API Consumers (Secondary):** Unauthenticated or authenticated users making requests to the protected downstream microservices. They do not interact with ShieldAPI directly, but experience its routing latency and rate-limiting responses.

2.4 Operating Environment

- **Platform:** Dockerized Linux environments (Alpine/Ubuntu).
- **Memory Store:** Redis 6.0 or higher.
- **Runtime Environments:** Python 3.10+ (Backend), Node.js (Frontend).
- **Deployment:** Containerized orchestration capable of running concurrent ASGI servers (Uvicorn).

2.5 Design and Implementation Constraints

- The gateway must remain stateless. Local memory cannot be used for rate-limiting variables to ensure horizontal scalability.
- Log analysis must be asynchronous to prevent blocking the main FastAPI thread.
- The system must minimize latency overhead; routing decisions must execute in milliseconds.

2.6 User Documentation

A standard README.md containing Docker deployment instructions and environment variable configurations (.env) will be provided alongside the repository.

2.7 Assumptions and Dependencies

- It is assumed the target backend microservice is fully functional and requires no code modifications to sit behind ShieldAPI.
- The deployment environment must have sufficient RAM to support Redis key-value caching.

3. External Interface Requirements

3.1 User Interfaces

The system shall provide a web-based dashboard for administrators. The dashboard shall follow a minimalist, high-contrast design system. It must display a real-time graph of incoming request volumes and a data table rendering currently blacklisted IP addresses.

3.2 Hardware Interfaces

No custom hardware interfaces are required. The software relies on standard Network Interface Cards (NICs) to handle TCP/IP traffic.

3.3 Software Interfaces

- **Target Backend Services:** ShieldAPI shall interface with downstream applications by forwarding HTTP requests and returning the exact HTTP responses generated by the target.
- **Redis Cache:** The application shall communicate with the Redis server via the RESP (Redis Serialization Protocol) on port 6379 to execute atomic increment operations.

3.4 Communications Interfaces

- All external incoming traffic shall be formatted as HTTP/HTTPS.
- The internal communication between the React frontend and the FastAPI backend shall utilize RESTful JSON API standards over HTTP.

4. System Features

4.1 Asynchronous Proxy Routing

4.1.1 Description and Priority

Priority: High. The core gateway must intercept client requests, read the target destination, and forward the payload without altering business logic.

4.1.2 Stimulus/Response Sequences

Stimulus: An external client sends a GET/POST request to the gateway URL. **Response:** The gateway validates the request, forwards it to the hidden backend, receives the backend's response, and transmits it back to the client.

4.1.3 Functional Requirements

- **REQ-1:** The gateway shall utilize asynchronous I/O handling to process concurrent connections without thread blocking.
- **REQ-2:** The system shall return a 502 Bad Gateway error if the downstream backend is unreachable.

4.2 Distributed Token-Bucket Rate Limiting

4.2.1 Description and Priority

Priority: High. To prevent volumetric attacks, the system restricts the number of requests an IP can make within a specified time window.

4.2.2 Stimulus/Response Sequences

Stimulus: A client initiates a request sequence exceeding the defined token limit (e.g., 60 requests per minute). **Response:** The system denies the overflow requests instantly and returns a 429 Too Many Requests status code.

4.2.3 Functional Requirements

- **REQ-3:** The rate limiter shall utilize Redis to track atomic state across all instances.
- **REQ-4:** Each request shall decrement the available token count for the originating IP address.
- **REQ-5:** The bucket shall replenish tokens automatically at the algorithmically defined rate.

4.3 Heuristic Anomaly Detection Engine (Anomaly Guardian)

4.3.1 Description and Priority

Priority: High. An active background process that streams logs to detect hostile activity and apply automatic network bans.

4.3.2 Stimulus/Response Sequences

Stimulus: An IP address triggers multiple 401 Unauthorized or 403 Forbidden errors within a 10-second window. **Response:** The engine updates the Redis blacklist cache, causing all subsequent traffic from that IP to be immediately dropped with a 403 Forbidden status.

4.3.3 Functional Requirements

- **REQ-6:** The engine shall parse access logs as a continuous data stream (Pipes and Filters).
- **REQ-7:** The engine shall evaluate event frequencies against a configurable heuristic threshold matrix.
- **REQ-8:** Blacklisted IPs shall remain blocked until the TTL (Time-To-Live) cache expires or an administrator manually revokes the ban via the dashboard.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The addition of the ShieldAPI layer shall not add more than 50 milliseconds of latency to a standard HTTP request under normal load conditions. Redis queries must execute in under 5 milliseconds to ensure optimal rate-limiter performance.

5.2 Safety Requirements

If the Redis instance crashes, the system should default to a "fail-open" state for routing, ensuring legitimate business traffic is not entirely halted by an infrastructure failure, while logging the incident heavily.

5.3 Security Requirements

The system must strip identifying backend server headers (e.g., X-Powered-By) before returning responses to the external client to prevent reconnaissance probing.

5.4 Software Quality Attributes

- **Maintainability:** The architecture must enforce strict separation of concerns. Security logic must never bleed into backend routing logic.
- **Scalability:** The architecture must allow multiple identical FastAPI containers to be spun up horizontally without causing state conflicts in the rate limiter.

5.5 Business Rules

Only authenticated users with system administrator privileges shall be permitted access to the MVC dashboard interface.

6. Other Requirements

The entire application stack (FastAPI, Redis, React, Dummy Backend) must be orchestratable via Docker Compose to guarantee environmental parity during the academic demonstration and deployment phases.

Appendix A: Glossary

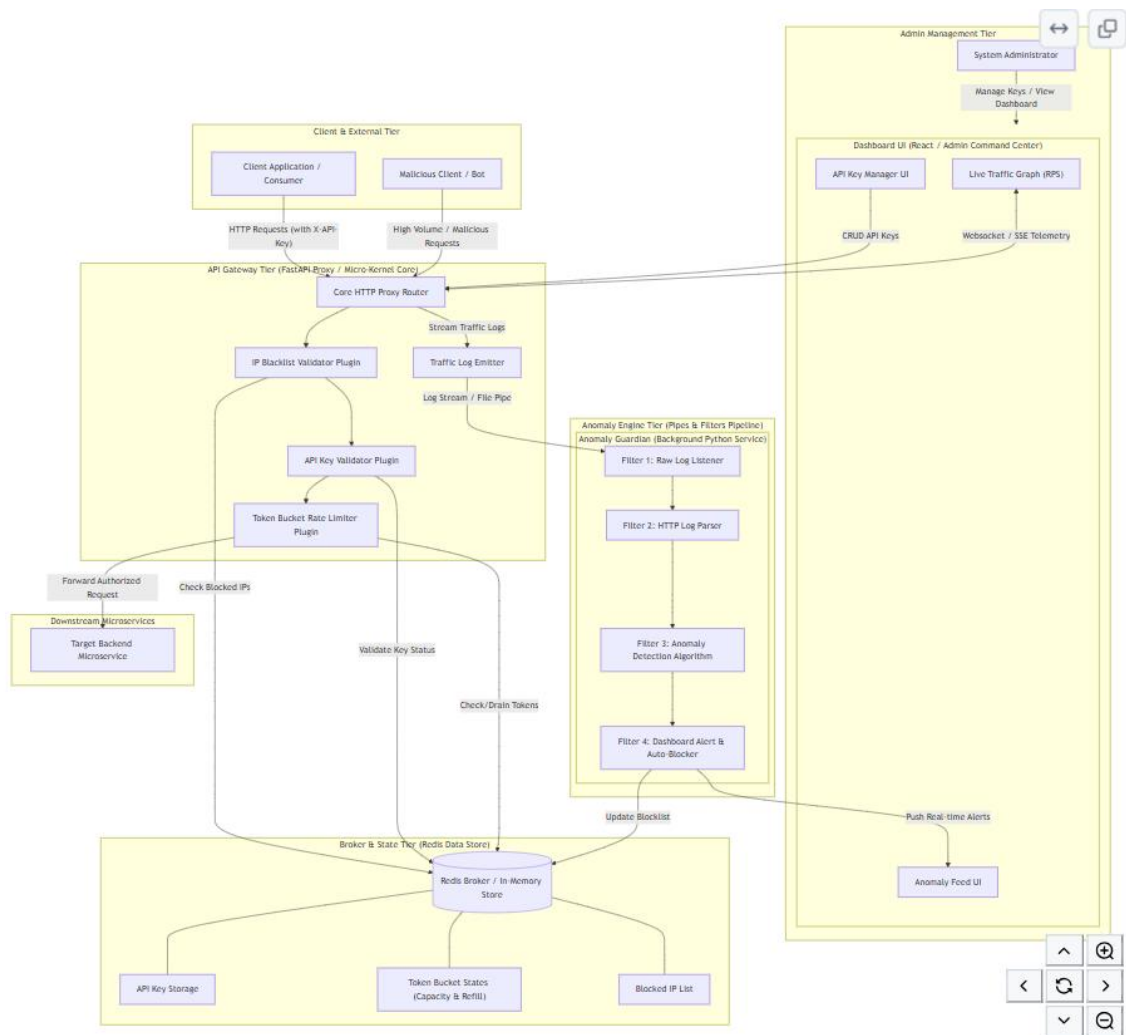
- **API Gateway:** A server that acts as an API front-end, receiving requests, enforcing throttling, and routing them to the back-end.
- **DDoS (Distributed Denial of Service):** A malicious attempt to disrupt normal traffic by overwhelming the target or its surrounding infrastructure with a flood of Internet traffic.
- **Token-Bucket:** An algorithm used in packet switched and telecommunications networks to dictate memory allocations and control data transmission rates.
- **Broker Pattern:** An architectural pattern that coordinates communication between decoupled components.
- **ASGI:** Asynchronous Server Gateway Interface.

Appendix B: Analysis Models

Component Diagram Layout Strategy for Diagramming Tools:

- **External Actor (Client):** Initiates HTTP Request.
- **Component 1 (FastAPI Gateway):** Receives request. Checks with Component 2.
- **Component 2 (Redis / Broker):** Checks Token Bucket and Blacklist state.
- **Component 3 (Backend Microservice):** Receives routed traffic if Component 2 approves.
- **Component 4 (Anomaly Guardian):** Asynchronously pulls error logs from Component 1, pushes new ban rules to Component 2.
- **Component 5 (React Dashboard / MVC):** Pulls state data from Component 2 for visual rendering.

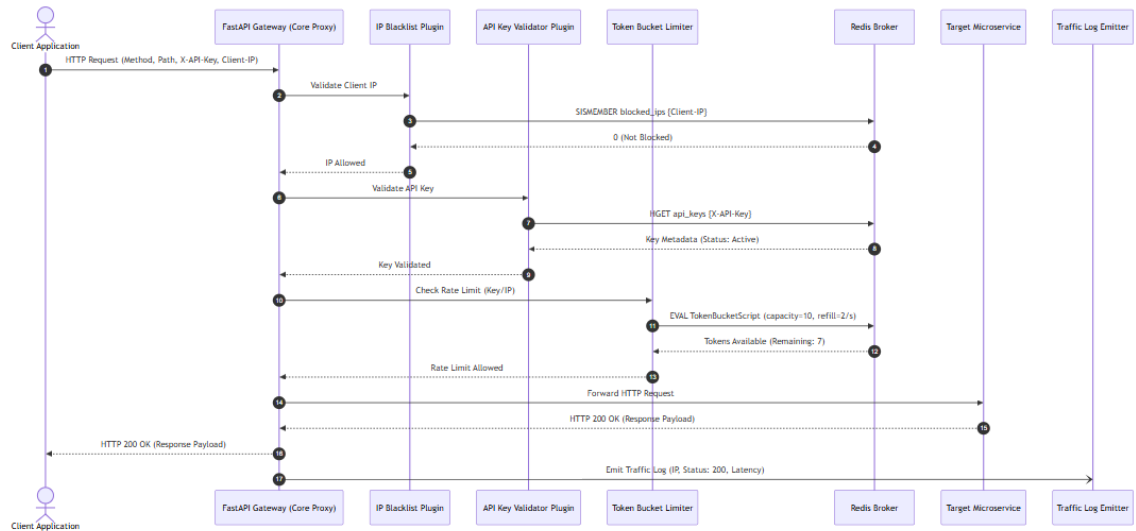
1. SYSTEM COMPONENT DIAGRAM:



2. Sequence Diagrams

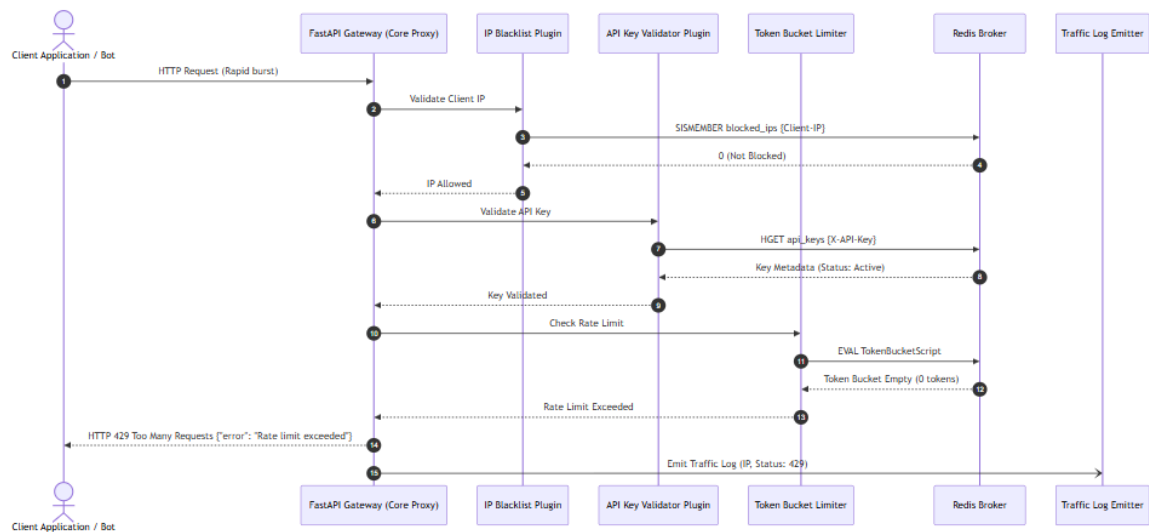
2.1 HTTP Request Lifecycle (Normal Authorized Request)

This diagram details the successful processing flow of an incoming client request passing through the IP blacklist check, API key validation, and token bucket rate limiter before reaching the downstream target microservice.



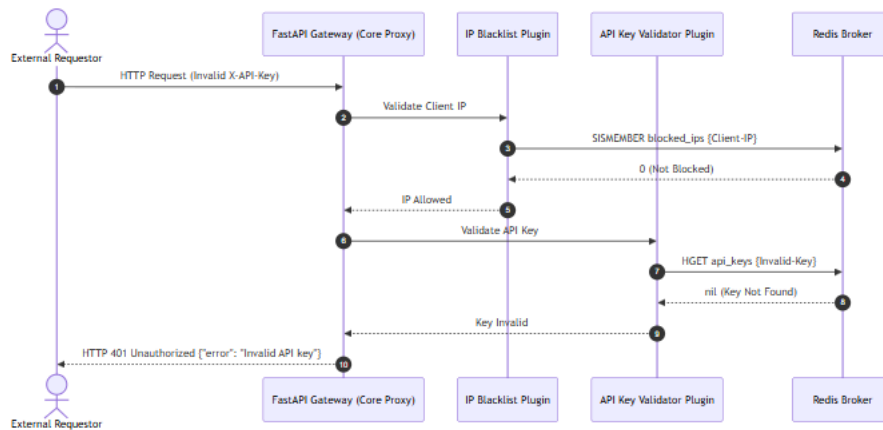
2.2 Rate Limited Request Flow (HTTP 429 Too Many Requests)

When a client exceeds their permitted request rate, the Token Bucket Limiter plugin detects an empty token bucket and immediately drops the request with HTTP 429.



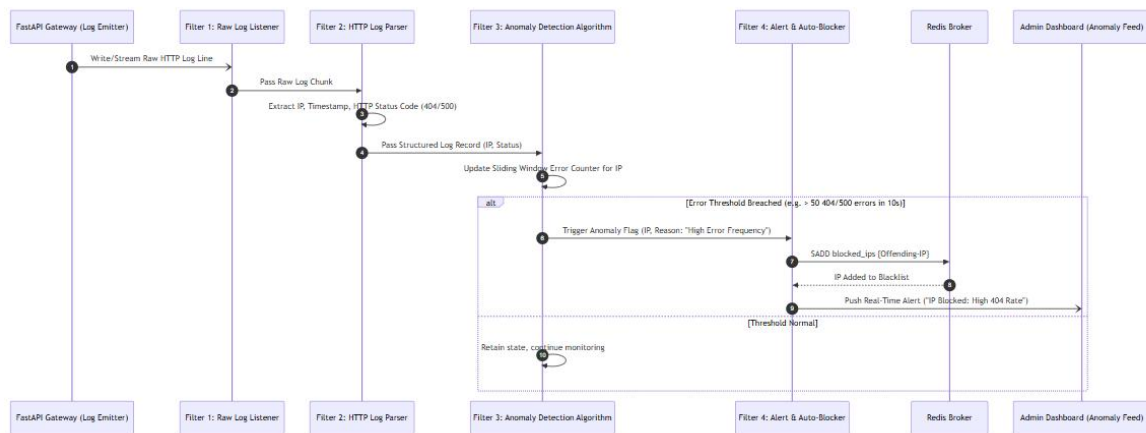
2.3 Invalid API Key Request Flow (HTTP 401 Unauthorized)

If an invalid or missing API key is provided, the API Key Validator plugin halts execution early.



2.4 Anomaly Guardian Processing Pipeline (Pipes & Filters)

This sequence maps how background traffic logs are processed sequentially by the Anomaly Guardian pipeline to detect abuse (excessive 404/500 errors) and auto-block offending IPs.



2.5 Admin API Key Management Flow

This sequence demonstrates how a system administrator interacts with the Admin Dashboard UI to generate or revoke API keys stored in Redis.

